Probability Basics: $p(x) = \int p(x, y)\, dy$

Bayes' rule: $p(x \mid y) = \frac{p(x,y)}{p(y)} = \frac{p(y|x)\, p(x)}{p(y)}$.

Independence: $X \perp Y \iff p(x, y) = p(x)p(y)$.

For i.i.d. $\mathcal{D} = \{x_1, \ldots, x_n\}$: $p(\mathcal{D}) = \prod_{i=1}^n p(x_i)$, $\log p(\mathcal{D}) = \sum_{i=1}^n \log p(x_i)$.

Expectation / Variance / Covariance:

$\mathbb{E}[X] = \int x\, p(x)\, dx$, $\mathbb{E}[g(X)] = \int g(x)\, p(x)\, dx$.

$\mathrm{var}(X) = \mathbb{E}[X^2] - \mathbb{E}[X]^2$

$\mathrm{cov}(X, Y) = \mathbb{E}[XY] - \mathbb{E}[X]\mathbb{E}[Y]$.

$\mathrm{var}(X + Y) = \mathrm{var}(X) + \mathrm{var}(Y) + 2\,\mathrm{cov}(X, Y)$.

If X, Y are independent, then $\mathbb{E}[XY] = \mathbb{E}[X]\mathbb{E}[Y]$.

Vector Random Variables: For $x = [x_1, \ldots, x_n]^\top$, $\mathbb{E}[x] = [\mathbb{E}[x_1], \ldots, \mathbb{E}[x_n]]^\top$, $\mathbb{E}[Ax] = A\mathbb{E}[x]$.

$\mathrm{cov}(x) = \Sigma_{xx} = \mathbb{E}[(x - \mathbb{E}[x])(x - \mathbb{E}[x])^\top]$.

$\mathrm{cov}(Ax) = A\,\mathrm{cov}(x)\, A^\top$, $\mathrm{cov}(x, y) = \Sigma_{xy} = \mathbb{E}[(x - \mathbb{E}[x])(y - \mathbb{E}[y])^\top]$.

Bernoulli Distribution: $X \sim \mathrm{Bern}(\theta)$, $x \in \{0, 1\}$, $p(x \mid \theta) = \theta^x (1 - \theta)^{1-x}$.

$\mathbb{E}[X] = \theta$, $\mathrm{var}(X) = \theta(1 - \theta)$.

Uniform Distribution: $X \sim \mathrm{Unif}(a, b)$

$p(x \mid a, b) = \frac{1}{b-a}$ for $a \le x \le b$, else 0.

$\mathbb{E}[X] = \frac{a+b}{2}$, $\mathrm{var}(X) = \frac{(b-a)^2}{12}$.

Exponential Distribution:

$X \sim \mathrm{Exp}(\lambda)$, $p(x \mid \lambda) = \lambda e^{-\lambda x}$ for $x \ge 0$, else 0. $\mathbb{E}[X] = \frac{1}{\lambda}$, $\mathrm{var}(X) = \frac{1}{\lambda^2}$.

Binomial Distribution:

$X \sim \mathrm{Bin}(n, \theta)$, $P(X = x) = \binom{n}{x} \theta^x (1 - \theta)^{n-x}$; $\binom{n}{x} = \frac{n!}{x!(n-x)!}$, $x = 0, \ldots, n$.

$\mathbb{E}[X] = n\theta$, $\mathrm{var}(X) = n\theta(1 - \theta)$.

Parametric Bayesian Model: Assume $x \sim p(x \mid \theta)$.

Then $p(\theta \mid x) = \frac{p(x|\theta)p(\theta)}{p(x)} \propto p(x \mid \theta)\, p(\theta)$, where posterior $\propto$ likelihood $\times$ prior. Normalization: $p(x) = \int p(x \mid \theta) p(\theta)\, d\theta$.

Conjugate Prior: If $p(\theta)$ and $p(\theta \mid x)$ have the same form, then $p(\theta)$ is a conjugate prior for $p(x \mid \theta)$.

$\log p(\theta \mid x) = \log p(x \mid \theta) + \log p(\theta) + \text{const}$.

Beta Distribution:

$\theta \sim \mathrm{Beta}(a, b)$, $a, b > 0$, $\theta \in [0, 1]$,

$p(\theta \mid a, b) = \frac{\Gamma(a+b)}{\Gamma(a)\Gamma(b)} \theta^{a-1} (1 - \theta)^{b-1}$.

Bernoulli + Beta:

1. For $x \in \{0, 1\}$, $p(\theta \mid x) \propto \mathrm{Bern}(x \mid \theta)\, \mathrm{Beta}(\theta \mid a, b) \propto \theta^{x+a-1} (1 - \theta)^{b-x}$.

Hence $\theta \mid x \sim \mathrm{Beta}(a + x,\ b + 1 - x)$.

2. For i.i.d. $\mathcal{D} = \{x_1, \ldots, x_n\}$,

$p(\mathcal{D} \mid \theta) = \prod_{i=1}^n p(x_i \mid \theta) = \prod_{i=1}^n \theta^{x_i} (1 - \theta)^{1-x_i}$.

$p(\theta \mid \mathcal{D}) \propto p(\mathcal{D} \mid \theta)\, p(\theta) \propto \theta^{\sum_{i=1}^n x_i + a - 1} (1 - \theta)^{n - \sum_{i=1}^n x_i + b - 1}$.

$\theta \mid \mathcal{D} \sim \mathrm{Beta}(a + \sum_{i=1}^n x_i,\ b + n - \sum_{i=1}^n x_i)$.

$\log p(\mathcal{D} \mid \theta) = \sum_{i=1}^n (x_i \log \theta + (1 - x_i) \log(1 - \theta))$.

Categorical Distribution:

$X \sim \mathrm{Cat}(\theta_1, \ldots, \theta_K)$, $\theta_k \ge 0$, $\sum_{k=1}^K \theta_k = 1$, $P(X = k) = \theta_k$. For i.i.d. $\mathcal{D} = \{x_1, \ldots, x_n\}$ and $N_k = \sum_{i=1}^n \mathbf{1}\{x_i = k\}$,

$p(\mathcal{D} \mid \theta) = \prod_k^K \theta_k^{N_k}$.

Dirichlet Distribution:

$\theta = (\theta_1, \ldots, \theta_K)$, $\theta_k \in [0, 1]$, $\sum_{k=1}^K \theta_k = 1$.

$p(\theta \mid \alpha) = \frac{\Gamma(\sum_{k=1}^K \alpha_k)}{\prod_{k=1}^K \Gamma(\alpha_k)} \prod_{k=1}^K \theta_k^{\alpha_k - 1}$.

Posterior with categorical counts:

$\theta \mid \mathcal{D} \sim \mathrm{Dir}(\alpha_1 + N_1, \ldots, \alpha_K + N_K)$. Beta is the special case $K = 2$.

Gaussian Distribution: $X \sim \mathcal{N}(\mu, \sigma^2)$ with

$$p(x \mid \mu, \sigma^2) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right).$$

Standard normal: $Z \sim \mathcal{N}(0, 1)$. Useful properties:

$aX \sim \mathcal{N}(a\mu, a^2\sigma^2)$, $X + c \sim \mathcal{N}(\mu + c, \sigma^2)$, $X = \sigma Z + \mu \sim \mathcal{N}(\mu, \sigma^2)$. If $X \sim \mathcal{N}(\mu, \sigma^2)$ and $Y \sim \mathcal{N}(\xi, \nu^2)$ are independent, then $X + Y \sim \mathcal{N}(\mu + \xi, \sigma^2 + \nu^2)$.

Jointly Gaussian: $X = [X_1, \ldots, X_n]^\top$ is jointly Gaussian iff $a^\top X = \sum_{i=1}^n a_i X_i$ is Gaussian for all $a \in \mathbb{R}^n$. If $Z_1, \ldots, Z_K \overset{i.i.d.}{\sim} \mathcal{N}(0, 1)$ and $X = AZ + \mu$, then $\mathbb{E}[X] = \mu$, $\mathrm{cov}(X) = AA^\top$.

$$\mathcal{N}(x \mid \mu, \Sigma) = \frac{1}{(2\pi)^{K/2} (\det \Sigma)^{1/2}} \exp\left(-\frac{1}{2}(x - \mu)^\top \Sigma^{-1} (x - \mu)\right).$$

$p(\mathbf{x} \mid \theta) = p(x_1 \mid \theta) p(x_2 \mid x_1, \theta)$.

2D Joint Gaussian: For $X = \begin{bmatrix} X \\ Y \end{bmatrix}$, $\mu = \begin{bmatrix} \mu_X \\ \mu_Y \end{bmatrix}$, $\Sigma = \begin{bmatrix} \sigma_X^2 & \mathrm{cov}(X, Y) \\ \mathrm{cov}(X, Y) & \sigma_Y^2 \end{bmatrix}$, the correlation coefficient is $\rho_{X,Y} = \frac{\mathrm{cov}(X,Y)}{\sigma_X \sigma_Y}$.

Likelihood Function: Given i.i.d. data $\mathcal{D} = \{x_1, \ldots, x_n\}$, the likelihood is $p(\mathcal{D} \mid \theta) = \prod_{i=1}^n p(x_i \mid \theta)$.

Likelihood Examples: If $x_i \overset{i.i.d.}{\sim} \mathrm{Bern}(\theta)$, then $p(\mathcal{D} \mid \theta) = \theta^{N_1} (1 - \theta)^{N_0}$, where $N_k = \sum_{i=1}^n \mathbf{1}\{x_i = k\}$. If $x_i \overset{i.i.d.}{\sim} \mathrm{Exp}(\lambda)$, then $p(\mathcal{D} \mid \lambda) = \lambda^n \exp(-\lambda \sum_{i=1}^n x_i)$. If $x_i \overset{i.i.d.}{\sim} \mathcal{N}(\mu, \sigma^2)$, then $p(\mathcal{D} \mid \mu, \sigma^2) = \frac{1}{(2\pi)^{n/2} \sigma^n} \exp\left(-\frac{1}{2\sigma^2} \sum_{i=1}^n (x_i - \mu)^2\right)$.

Maximum Likelihood Estimate (MLE):

$\theta_{\mathrm{ML}} = \arg\max_\theta p(\mathcal{D} \mid \theta) = \arg\max_\theta \log p(\mathcal{D} \mid \theta)$.

For i.i.d. data, $\log p(\mathcal{D} \mid \theta) = \sum_{i=1}^n \log p(x_i \mid \theta)$.

Bernoulli MLE: If $p(\mathcal{D} \mid \theta) = \theta^{N_1} (1 - \theta)^{N_0}$, then $\log p(\mathcal{D} \mid \theta) = N_1 \log \theta + N_0 \log(1 - \theta)$, and $\theta_{\mathrm{ML}} = \frac{N_1}{N_0 + N_1} = \frac{N_1}{n}$.

Exponential MLE: If $x_i \overset{i.i.d.}{\sim} \mathrm{Exp}(\lambda)$, then $\log p(\mathcal{D} \mid \lambda) = n \log \lambda - \lambda \sum_{i=1}^n x_i$, and $\lambda_{\mathrm{ML}} = \frac{n}{\sum_{i=1}^n x_i}$.

Gaussian Mean MLE (known σ^2): If $x_i \overset{i.i.d.}{\sim} \mathcal{N}(\mu, \sigma^2)$ with known variance, then $\log p(\mathcal{D} \mid \mu) = -\frac{1}{2\sigma^2} \sum_{i=1}^n (x_i - \mu)^2 + \text{const}$, and $\mu_{\mathrm{ML}} = \frac{1}{n} \sum_{i=1}^n x_i$.

KL Divergence (supplementary):

$D(p \| q) = \mathbb{E}_p\left[\log \frac{p(x)}{q(x)}\right] \ge 0$, with equality iff $p = q$ almost everywhere. For large n, MLE approximately minimizes $D(p_{\theta_0} \| p_\theta)$.

Linear Regression Model:

$y = w^\top x + \epsilon$, $\epsilon \sim \mathcal{N}(0, \sigma^2)$, so $p(y \mid x, w) = \mathcal{N}(y \mid w^\top \phi(x), \sigma^2)$. With basis expansion, $\phi(x) = [\phi_1(x), \ldots, \phi_M(x)]^\top$, and $y = w^\top \phi(x) + \epsilon$.

MLE for Linear Regression Weights: Given $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^n$, $\log p(y \mid \phi(x), w) = \sum_{i=1}^n \log \mathcal{N}(y_i \mid w^\top \phi(x_i), \sigma^2) = -\frac{1}{2\sigma^2} \sum_{i=1}^n (y_i - w^\top \phi(x_i))^2 + \text{const}$. Let

$$\Phi = \begin{bmatrix} \phi(x_1)^\top \\ \vdots \\ \phi(x_n)^\top \end{bmatrix} \text{ and } y = \begin{bmatrix} y_1 \\ \vdots \\ y_n \end{bmatrix}. \text{ Then}$$

$\log p(y \mid \Phi, w) = -\frac{1}{2\sigma^2} \|\Phi w - y\|^2 + \text{const}$, so MLE is least squares: $\min_w \|\Phi w - y\|^2$, and $w_{\mathrm{ML}} = (\Phi^\top \Phi)^{-1} \Phi^\top y$ if $\Phi^\top \Phi$ is invertible.

Example Basis Functions: For $x = [x_1, x_2]^\top$, $\phi(x) = [1,\ x_1,\ x_2,\ x_1^2,\ x_2^2]^\top$, so $y = w_1 + w_2 x_1 + w_3 x_2 + w_4 x_1^2 + w_5 x_2^2 + \epsilon$.

Goodness of Fit: $\mathrm{RSS} = \sum_{i=1}^n (y_i - \hat{y}_i)^2$.

$\mathrm{RMSE} = \sqrt{\frac{1}{n} \mathrm{RSS}}$. $\bar{y} = \frac{1}{n} \sum_{i=1}^n y_i$.

$$R^2 = 1 - \frac{\mathrm{RSS}}{\mathrm{TSS}} = 1 - \frac{\mathrm{RSS}}{\sum_{i=1}^n (y_i - \bar{y})^2}.$$

Maximum A Posteriori (MAP):

$\theta_{\mathrm{MAP}} = \arg\max_\theta p(\theta \mid \mathcal{D})$. Using Bayes' rule, $p(\theta \mid \mathcal{D}) \propto p(\mathcal{D} \mid \theta) p(\theta)$, and $\log p(\theta \mid \mathcal{D}) = \log p(\mathcal{D} \mid \theta) + \log p(\theta) + \text{const}$.

Bernoulli + Beta MAP: If $p(\mathcal{D} \mid \theta) = \theta^{N_1} (1 - \theta)^{N_0}$ and $p(\theta) = \mathrm{Beta}(a, b)$, then $\log(p(\mathcal{D} \mid \theta) p(\theta)) = (N_1 + a - 1) \log \theta + (N_0 + b - 1) \log(1 - \theta)$, and $\theta_{\mathrm{MAP}} = \frac{N_1 + a - 1}{n + a + b - 2}$.

Gaussian Mean MAP (known σ^2): If $x_i \overset{i.i.d.}{\sim} \mathcal{N}(\mu, \sigma^2)$ and $\mu \sim \mathcal{N}(\mu_0, 1)$, then $\log p(\mu \mid \mathcal{D}) = -\frac{1}{2\sigma^2} \sum_{i=1}^n (x_i - \mu)^2 - \frac{1}{2}(\mu - \mu_0)^2 + \text{const}$, and $\mu_{\mathrm{MAP}} = \frac{\frac{1}{\sigma^2} \sum_{i=1}^n x_i + \mu_0}{\frac{n}{\sigma^2} + 1}$.

MAP Classification Rule: Choose $\delta(x) = \arg\max_k p(y = k \mid x)$. This minimizes classification error for the given x.

Naive Bayes: Assume conditional independence of features given the class:

$p(x \mid y = c, \theta) = \prod_{d=1}^D p(x_d \mid \theta_{dc})$. If $\pi(c) = p(y = c)$ is the class prior, then

$p(y = c \mid x, \theta) \propto \pi(c) \prod_{d=1}^D p(x_d \mid \theta_{dc})$. For binary features, $x_d \mid y = c \sim \mathrm{Bern}(\theta_{dc})$.

EM Algorithm: 用于处理含有 latent variables / missing data 的 MLE 问题。Observed data: $x = \{x_1, \ldots, x_n\}$.; Latent variables: $z = \{z_1, \ldots, z_n\}$.; Parameters: θ.

目标是求 MLE: $\hat{\theta} = \arg\max_\theta \log p(x \mid \theta)$. 但 z 没有被观测到，难直接最大化。

Observed-data Likelihood: 若 latent label z_i unknown，则对所有 possible components 求和: $p(x_i \mid \Theta) = \sum_{k=1}^K \pi_k f_k(x_i \mid \theta_k)$, $\Theta = \{\pi_k, \theta_k\}_{k=1}^K$. For independent samples,

$\ell(\Theta) = \sum_{i=1}^n \log\left[\sum_{k=1}^K \pi_k f_k(x_i \mid \theta_k)\right]$. 这是 log-sum form，通常难以直接 optimize，因此用 EM.

E-step: Q and Responsibilities: At iteration m, current parameters are $\Theta^{(m)}$.

General E-step:

$Q(\Theta \mid \Theta^{(m)}) = \mathbb{E}_{z|x,\Theta^{(m)}}[\log p(x, z \mid \Theta)]$.

For a mixture model: $Q(\Theta \mid \Theta^{(m)}) = \sum_{i=1}^n \sum_{k=1}^K r_{ik}^{(m)} [\log \pi_k + \log f_k(x_i \mid \theta_k)]$.

Responsibilities: $r_{ik}^{(m)} = P(z_i = k \mid x_i, \Theta^{(m)}) = \frac{\pi_k^{(m)} f_k(x_i|\theta_k^{(m)})}{\sum_{j=1}^K \pi_j^{(m)} f_j(x_i|\theta_j^{(m)})}$.

其中 $r_{ik}^{(m)}$ 是 sample x_i 属于 component k 的 posterior probability，且 $\sum_{k=1}^K r_{ik}^{(m)} = 1$.

注意：E-step only updates $r_{ik}^{(m)}$，not parameters.

M-step: M-step 用 E-step 得到的 responsibilities 作为 soft labels 来更新 parameters:

$\Theta^{(m+1)} = \arg\max_\Theta Q(\Theta \mid \Theta^{(m)})$.

Effective sample size: $n_k^{(m)} = \sum_{i=1}^n r_{ik}^{(m)}$.

Mixture weight update: $\pi_k^{(m+1)} = \frac{n_k^{(m)}}{n}$.

Component parameter update: $\theta_k^{(m+1)} = \arg\max_{\theta_k} \sum_{i=1}^n r_{ik}^{(m)} \log f_k(x_i \mid \theta_k)$.

General calculation method:

$Q_k(\theta_k) = \sum_{i=1}^n r_{ik}^{(m)} \log f_k(x_i \mid \theta_k)$,

$\frac{\partial Q_k}{\partial \theta_k} = 0 \Rightarrow \theta_k^{(m+1)}$.

不同 distribution 只改变 $f_k(x_i \mid \theta_k)$，因此 M-step update 不同: Bernoulli $\Rightarrow$ update probability，Gaussian $\Rightarrow$ update mean/covariance，Poisson $\Rightarrow$ update rate.

Bernoulli Mixture: 若每个 component 是 Bernoulli distribution:

$f_k(x_i \mid \theta_k) = \theta_k^{x_i} (1 - \theta_k)^{1 - x_i}$, $x_i \in \{0, 1\}$.

E-step 计算 responsibility:

$r_{ik}^{(m)} = \frac{\pi_k^{(m)} (\theta_k^{(m)})^{x_i} (1 - \theta_k^{(m)})^{1 - x_i}}{\sum_{j=1}^K \pi_j^{(m)} (\theta_j^{(m)})^{x_i} (1 - \theta_j^{(m)})^{1 - x_i}}$.

M-step update:

$n_k^{(m)} = \sum_{i=1}^n r_{ik}^{(m)}$, $\pi_k^{(m+1)} = \frac{n_k^{(m)}}{n}$, $\theta_k^{(m+1)} = \frac{\sum_{i=1}^n r_{ik}^{(m)} x_i}{\sum_{i=1}^n r_{ik}^{(m)}}$.

这里 θ_k 是 component k 的 success probability；$\sum_i r_{ik}^{(m)} x_i$ 是 soft number of successes，$\sum_i r_{ik}^{(m)}$ 是 soft total count.

EM for GMM: $r_{ik}^{(m)} = p(z_i = k \mid x_i, \theta^{(m)}) = \frac{\pi_k^{(m)} \mathcal{N}(x_i|\mu_k^{(m)}, \Sigma_k^{(m)})}{\sum_{j=1}^K \pi_j^{(m)} \mathcal{N}(x_i|\mu_j^{(m)}, \Sigma_j^{(m)})}$.

$\pi_k^{(m+1)} = \frac{n_k^{(m)}}{n}$, $\mu_k^{(m+1)} = \frac{\sum_{i=1}^n r_{ik}^{(m)} x_i}{n_k^{(m)}}$,

$\Sigma_k^{(m+1)} = \frac{1}{n_k^{(m)}} \sum_{i=1}^n r_{ik}^{(m)} (x_i - \mu_k^{(m+1)})(x_i - \mu_k^{(m+1)})^\top$.

- Stop when $|\ell(\Theta^{(m+1)}) - \ell(\Theta^{(m)})| \le \epsilon$ or parameters become stable.

- EM makes likelihood non-decreasing: $\ell(\Theta^{(m+1)}) \ge \ell(\Theta^{(m)})$. But it may converge to a local maximum and depends on initialization.

K-Means as a Special Case of GMM: K-Means 可以看成 GMM 的一个特殊情形：设所有 components 都满足 $\Sigma_k = \sigma^2 I$, $\pi_k = \frac{1}{K}$，且它们是 fixed and known，因此只需要估计 cluster centers $\mu_1, \ldots, \mu_K$。

在 E-step 中，采用 hard assignment：对每个 sample x_i，找到最近的 center $k_i = \arg\min_k \|x_i - \mu_k^{(m)}\|^2$，并将 x_i 完全分配给该 cluster。Then $Q(\theta \mid \theta^{(m)}) = -\frac{1}{2\sigma^2} \sum_{i=1}^n \|x_i - \mu_{k_i}^{(m)}\|^2 + \text{const}$.

在 M-step 中，更新每个 center 为分配到该 cluster 的样本均值: $\mu_k^{(m+1)} = \frac{1}{N_k} \sum_{i:k_i=k} x_i$. where $N_k = \sum_{i=1}^n \mathbf{1}\{k_i = k\}$.

本质：*Assign each sample to the nearest center, then update each center by the mean of its assigned samples.*

MAP for Mixture Models: To regularize and avoid near-singular covariance matrices, impose a prior $p(\theta)$. 在 MLE 的基础上额外加入一个 prior term，用来避免参数走向过于极端的解。

Then $p(\theta \mid x) = \frac{p(x|\theta)p(\theta)}{p(x)}$, and

$\theta_{\mathrm{MAP}} = \arg\max_\theta (\log p(x \mid \theta) + \log p(\theta))$.

EM for MAP: E-step:

$Q(\theta \mid \theta^{(m)}) = \int p(y \mid x, \theta^{(m)}) \log p(y \mid \theta)\, dy$.

M-step: 在最大化 Q 的同时加入 prior penalty

$\theta^{(m+1)} = \arg\max_\theta (Q(\theta \mid \theta^{(m)}) + \log p(\theta))$. As the data dimension increases, the number of parameters also grows rapidly, and MLE is more likely to encounter singular matrices or numerical issues. In contrast, MAP estimation can suppress covariance collapse through the prior and reduce the risk of EM failure.

Transition Matrix

- Each row of $\mathbf{T}$ sums to one: $\sum_{j=1}^M T(i, j) = \sum_{j=1}^M p_{x[t]|x[t-1]}(j \mid i) = 1$. $\mathbf{T}$ is a (row) stochastic matrix.
- Given initial distribution $\mathbf{p}_0 = [p_0(1), p_0(2), \ldots, p_0(M)]$, $\mathbf{p}_1 = \mathbf{p}_0 \mathbf{T}$. For general t, let $\mathbf{p}_t = [p_t(1), \ldots, p_t(M)]$, then: $\mathbf{p}_t = \mathbf{p}_0 \mathbf{T}^t$.

MLE for Transition Matrix

- Estimate prior π and transition matrix $\mathbf{T}$ from training data: $p(x[0], \ldots, x[t] \mid \pi, \mathbf{T}) = \pi(x[0]) \mathbf{T}(x[0], x[1]) \ldots \mathbf{T}(x[t-1], x[t])$. Given n observed sequences $\mathcal{D} = \{\mathbf{x}_1[0 : t_1], \ldots, \mathbf{x}_n[0 : t_n]\}$, each of varying length $t_i + 1$, assume all data points follow the same $\mathbf{T}$.
- $\log p(\mathcal{D} \mid \pi, \mathbf{T}) = \sum_{i=1}^n \log \pi(x_i[0]) + \sum_{i=1}^n \sum_{t=1}^{t_i} \log \mathbf{T}(x_i[t-1], x_i[t])$
$= \sum_{x=1}^M N_x \log \pi(x) + \sum_{x=1}^M \sum_{y=1}^M N_{xy} \log \mathbf{T}(x, y)$.
$N_x = \sum_{i=1}^n \mathbb{1}(x_i[0] = x)$, $N_{xy} = \sum_{i=1}^n \sum_{t=1}^{t_i} \mathbb{1}(x_i[t-1] = x, x_i[t] = y)$.
- $\hat{\pi}(x) = \frac{N_x}{n}$, $\hat{\mathbf{T}}(x, y) = \frac{N_{xy}}{\sum_{z=1}^M N_{xz}}$.
- May predict certain strings are impossible if data have zero count of certain states (this is a form of overfitting).

HMM: Hidden Markov Model

A hidden Markov model consists of:

- A discrete state Markov chain with **hidden states** or latent variables $z[t] \in \{1, \ldots, M\}$, $t = 0, 1, \ldots$, with initial pdf π and transition matrix $\mathbf{T}$.
- An observation model with emission probabilities $p(\mathbf{x}[t] \mid z[t]) = p(\mathbf{x}[t] \mid \phi_{z[t]})$, where $\phi = (\phi_1, \ldots, \phi_M)$.

Baum-Welch Algorithm for HMM Training

1. Initialize $\theta^{(0)}$.
2. **E step**: Use Forward-Backward Algorithm to compute $\gamma_{i,t}(z) = p(z_i[t] = z \mid \mathbf{x}_i[0 : t_i], \theta^{(m)}) \propto \alpha_j(z) \beta_j(z)$. $\xi_{i,t}(z, z') = p(z_i[t-1] = z, z_i[t] = z' \mid \mathbf{x}_i[0 : t_i], \theta^{(m)}) \propto \alpha_{t-1}(z) p(\mathbf{x}_i[t] \mid z_i[t] = z') \beta_t(z') p(z_i[t] = z' \mid z_i[t-1] = z)$.
3. **M step**: Update parameters $\hat{\pi}(z) = \frac{\sum_{i=1}^n \gamma_{i,0}(z)}{n}$, $\hat{T}(z, z') = \frac{\sum_{i=1}^n \sum_{t=1}^{t_i} \xi_{i,t}(z, z')}{\sum_u \sum_{i=1}^n \sum_{t=1}^{t_i} \xi_{i,t}(z, u)}$, $\hat{\phi}_z =$ emission probability parameters. 4. Repeat E and M steps.

Inference in HMMs

- Filtering: Estimate latent state $p(z[t] \mid \mathbf{x}[0 : t])$ using observations up to time t (Forward Algorithm).
- Smoothing: Estimate $p(z[t] \mid \mathbf{x}[0 : T])$, using both past and future observations (Forward-Backward Algorithm).
- Fixed-lag smoothing: Estimate $p(z[t - l] \mid \mathbf{x}[0 : t])$ for online inference (Forward-Backward Algorithm).
- Prediction: Estimate $p(z[t + h] \mid \mathbf{x}[0 : t])$, where $h > 0$ is the prediction horizon: $p(z[t+h] \mid \mathbf{x}[0 : t]) = \sum_{z[t], \ldots, z[t+h-1]} p(z[t+h] \mid z[t+h-1]) p(z[t+h-1] \mid z[t+h-2]) \cdots p(z[t+1] \mid z[t]) \cdot p(z[t] \mid \mathbf{x}[0 : t])$.
- MAP sequence: using Viterbi Algorithm, most probable sequence $\mathbf{z}^*[0 : T] = \arg\max_{z[0:T]} p(z[0 : T] \mid \mathbf{x}[0 : T])$.

Monte Carlo Approximation:

If $x_1, \ldots, x_n \overset{i.i.d.}{\sim} p(x \mid \theta)$, then by SLLN $\mathbb{E}[f(x) \mid \theta] \approx \frac{1}{n} \sum_{i=1}^n f(x_i)$.

Inverse-CDF Sampling: 若目标分布的 CDF 为 F，可先取 $U \sim \mathrm{Unif}(0, 1)$，令 $U = F(X)$；$X = F^{-1}(U)$；now $P(X \le x) = F(x)$，so X 服从目标分布。

CDF Pseudo-Inverse: CDF 只要求单调不减，未必是一一对应；若 F 不可直接求逆，找"最小的使累计概率达到 u 的 x"。$F^{-1}(u) = \inf\{x : F(x) \ge u\}$.

Examples: 若 $X \sim \mathrm{Exp}(\lambda)$，$F(x) = 1 - e^{-\lambda x}$，则 $X = -\frac{1}{\lambda} \log(1 - U)$. 若 $X \sim \mathrm{Bern}(\theta)$，则 $X = \mathbf{1}\{U > 1 - \theta\}$.

Transformation Formula: If $Y = f(X)$ and the solutions to $f(x) = y$ are $x_1, \ldots, x_K$, then

$p_Y(y) = \sum_{k=1}^K \frac{p_X(x_k)}{|f'(x_k)|}$.

Examples: If $Y = aX + b$, then

$p_Y(y) = \frac{1}{|a|} p_X\left(\frac{y-b}{a}\right)$. If $Y = X^2$, then

$p_Y(y) = \frac{1}{2\sqrt{y}} (p_X(\sqrt{y}) + p_X(-\sqrt{y}))$.

Gamma Sampling for Integer Shape: 若 $Y \sim \mathrm{Gamma}(a, b)$ 且 a 为正整数，则 $Y = \sum_{i=1}^a X_i$，其中 $X_1, \ldots, X_a \overset{i.i.d.}{\sim} \mathrm{Exp}(b)$. 直觉上: $\mathrm{Gamma}(a, b)$ 是"等到第 a 次事件发生的总等待时间"。

Rejection Sampling Setup: 想从目标分布 $p(z)$ 中采样，但不能直接计算，只知道它和某个未归一化函数 $\tilde{p}(z)$ 成正比 $p(z) = \frac{1}{M} \tilde{p}(z)$, where M is unknown. Choose proposal $q(z)$ and constant k such that $kq(z) \ge \tilde{p}(z)$ for all z, with $\mathrm{supp}(p) \subset \mathrm{supp}(q)$.

Rejection Sampling Algorithm: 1. Sample $z \sim q(\cdot)$. 2. Sample $u \sim \mathrm{Unif}[0, kq(z)]$. 3. Accept z if $u \le \tilde{p}(z)$.

Acceptance Probability:

$P(\text{accept}) = \int \frac{\tilde{p}(z)}{kq(z)} q(z)\, dz = \frac{M}{k}$. Hence we want k as small as possible subject to $kq(z) \ge \tilde{p}(z)$.

Why Rejection Sampling Works: Accepted samples have cdf $P(z \le z_0 \mid \text{accepted}) = \frac{1}{M} \int_{z \le z_0} \tilde{p}(z)\, dz$, so accepted z is distributed according to $p(z)$.

Rejection Sampling for Bayesian Inference: For posterior $p(\theta \mid \mathcal{D}) \propto p(\mathcal{D} \mid \theta) p(\theta)$, take $\tilde{p}(\theta) = p(\mathcal{D} \mid \theta) p(\theta)$. If choose proposal $q(\theta) = p(\theta)$, then

$k = \max_\theta \frac{\tilde{p}(\theta)}{q(\theta)} = \max_\theta p(\mathcal{D} \mid \theta)$, i.e. the maximum likelihood value. 先从 prior 采样作为 proposal，再用 likelihood 决定是否 accept；likelihood 越大，越容易被 accept

Importance Sampling:

不直接从 $p(z)$ 采样，而是从一个容易采样的 proposal distribution $q(z)$ 采样。用 weight 修正 $q(z)$ 和 $p(z)$ 之间的差异。

因为：$\mathbb{E}_p[f(z)] = \int f(z) p(z)\, dz, = \int f(z) \frac{p(z)}{q(z)} q(z)\, dz. = \mathbb{E}_q\left[f(z) \frac{p(z)}{q(z)}\right]$.

定义 importance weight：$w(z) = \frac{p(z)}{q(z)}$.

于是如果 $z_1, \ldots, z_n \sim q(z)$，则有：

$\mathbb{E}_p[f(z)] \approx \frac{1}{n} \sum_{i=1}^n w(z_i) f(z_i)$,

直观理解：从 q 采样，用 $\frac{p}{q}$ 修正采样偏差。

如果某个区域 $p(z)$ 大但 $q(z)$ 小，说明这个区域在 target distribution 中很重要，但 proposal distribution 很少采到，所以采到后要给更大的 weight。

Support Condition:

Importance Sampling 要求 proposal distribution $q(z)$ 覆盖 target distribution $p(z)$ 重要的区域。$\mathrm{supp}(p) \subseteq \mathrm{supp}(q)$.

如果我们要估计：$\mathbb{E}_p[f(z)]$，则需要：

$\mathrm{supp}(f(z)p(z)) \subseteq \mathrm{supp}(q)$.

只要某个地方：$f(z)p(z) \ne 0$，就必须有：$q(z) > 0$.

否则，如果某个区域 $p(z) > 0$ 但 $q(z) = 0$，那么从 $q(z)$ 永远采不到这个区域

Self-normalized Importance Sampling:

有时 target density $p(z)$ 不是直接已知的，而是只知道一个 unnormalized density：$\tilde{p}(z)$.，

$p(z) = \frac{\tilde{p}(z)}{Z}$，其中：$Z = \int \tilde{p}(z)\, dz$ 是 normalization constant。

如果 Z 不知道，就不能直接计算：

$w(z) = \frac{p(z)}{q(z)} = \frac{\tilde{p}(z)}{Zq(z)}$., Z unknown。

因此改用 unnormalized importance weight：

$\tilde{w}(z) = \frac{\tilde{p}(z)}{q(z)}$.

然后用 Self-normalized Importance Sampling estimator：

$\mathbb{E}_p[f(z)] \approx \frac{\sum_{i=1}^n \tilde{w}(z_i) f(z_i)}{\sum_{i=1}^n \tilde{w}(z_i)}$, $z_i \sim q(z)$.

Rare-event Probability Estimation: 想估计一个 p(rare-event) 可以选择一个 proposal distribution q，让它要容易采到 tail region (a, ∞)，提高估计效率。

eg:$P(X > a)$. 如果直接从 p 采样，可能很少采到 $X > a$ 的样本，导致 estimator variance 很大。

因为：$P(X > a) = \mathbb{E}_p\left[\mathbf{1}_{\{X > a\}}\right] = \int \mathbf{1}_{\{z > a\}} p(z)\, dz$.

使用 proposal distribution $q(z)$ 后：

$P(X > a) = \int \mathbf{1}_{\{z > a\}} \frac{p(z)}{q(z)} q(z)\, dz \approx \frac{1}{n} \sum_{i=1}^n \mathbf{1}_{\{z_i > a\}} \frac{p(z_i)}{q(z_i)}$, $z_i \sim q(z)$. q 的取值覆盖 >a 那段 tail region 就可以

Sampling Importance Resampling (SIR):

Importance Sampling 得到的是 weighted samples：$(z_i, \tilde{w}_i), i = 1, \ldots, n$.

SIR 的作用是：用 normalized weights $\bar{w}_i$ 来 resampling

步骤如下：

1.Sample proposal samples $z_1, \ldots, z_n \sim q(z)$，$q(z)$ 易采样

2.Compute unnormalized importance weights for each sample: $\tilde{w}_i = \frac{\tilde{p}(z_i)}{q(z_i)}$

3.Normalize the weights，使和 =1: $\bar{w}_i = \frac{\tilde{w}_i}{\sum_{j=1}^n \tilde{w}_j}$，因此 $\sum_{i=1}^n \bar{w}_i = 1$，$\bar{w}_i$ 可以作 resampling probabilities.

4.Resample with replacement from $\{z_1, \ldots, z_n\}$ according to probabilities $\{\bar{w}_1, \ldots, \bar{w}_n\}$，得到

resampled points $z_1^*, \dots, z_m^*$。Resampling 后得到的样本近似服从 target distribution $p(z)$。当 $n \to \infty$ 时，resampled points 的 empirical distribution 会收敛到 $p(z)$。

SIR for Bayesian Inference:
$p(\theta \mid \mathcal{D}) \propto p(\mathcal{D} \mid \theta) p(\theta)$.
因此 unnormalized posterior density 是：
$\tilde{p}(\theta) = p(\mathcal{D} \mid \theta) p(\theta)$.
如果选择 prior distribution 作为 proposal distribution：
$q(\theta) = p(\theta)$，那么 importance weight 为：
$\tilde{w}(\theta_i) = \frac{\tilde{p}(\theta_i)}{q(\theta_i)} = \frac{p(\mathcal{D} \mid \theta_i) p(\theta_i)}{p(\theta_i)} = p(\mathcal{D} \mid \theta_i)$.
Normalized weights 为：$\bar{w}_i = \frac{p(\mathcal{D} \mid \theta_i)}{\sum_{j=1}^n p(\mathcal{D} \mid \theta_j)}$.
SIR for Bayesian inference 的步骤是：1.Sample $\theta_1, \dots, \theta_n \sim p(\theta)$. 2.Compute weights: $\tilde{w}_i = p(\mathcal{D} \mid \theta_i)$. 3.Normalize the weights. 4.Resample θ_i according to the normalized weights.
Resampled θ values 可以看作 approximate samples from posterior distribution: $p(\theta \mid \mathcal{D})$.

Monte Carlo EM:
在 EM algorithm 中，E-step 需要计算：
$Q(\theta \mid \theta^{(m)}) = \mathbb{E}_{p(z \mid x, \theta^{(m)})} [\log p(x, z \mid \theta)] = \int p(z \mid x, \theta^{(m)}) \log p(x, z \mid \theta) \, dz$.
若积分无法解析计算，用 Monte Carlo approximation。从 latent variable posterior 中采样：
$z_1, \dots, z_n \sim p(z \mid x, \theta^{(m)})$.
然后：$Q(\theta \mid \theta^{(m)}) \approx \frac{1}{n} \sum_{i=1}^n \log p(x, z_i \mid \theta)$.
如果我们能直接从 $p(z \mid x, \theta^{(m)})$ 采样，那么这是普通 Monte Carlo approximation。如果不能直接采样，也可以引入 proposal distribution $q(z)$，再用 Importance Sampling 近似这个 E-step expectation。

MCMC Goal: 希望从高维的 target pdf $\pi(x)$ 中进行采样。构造一个 Markov chain $Z_0, Z_1, \dots$，使得当 $n \to \infty$ 时，Z_n 的分布趋近于 $\pi(\cdot)$。

Stationary Distribution: 对于一个具有 transition probability $T(x, y)$ 的 homogeneous Markov chain，若 $\sum_x \pi(x) T(x, y) = \pi(y)$ 对所有 y 都成立，则 π 是 stationary distribution。Matrix form: $\pi T = \pi$.

Asymptotic Convergence: 如果该 chain 是 irreducible、positive recurrent 且 aperiodic，那么 $\pi_k = \pi_0 T^k \to \pi$ 当 $k \to \infty$ 时成立，其中 π 是 stationary distribution。这样的 chain 是 ergodic。(遍历的)

Reversibility: A sufficient condition for π to be stationary is $\pi(x) T(x, y) = \pi(y) T(y, x) \forall x, y$. 在平稳状态下，从 x 到 y 的概率流量，等于从 y 到 x 的概率流量

Burn-In: Discard the initial part of the chain before it is close to stationarity. Typical burn-in: first 1000 to 5000 samples.

Thinning: MCMC samples are usually correlated, especially consecutive samples. Thinning is used to reduce autocorrelation among the retained samples, making them more nearly independent.

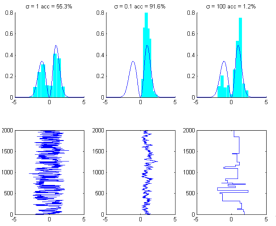
Metropolis–Hastings (MH): 假设 target 只知道到 normalization 常数为止：$\pi(x) \propto \tilde{\pi}(x)$. 选择 proposal $q(x, y)$。在第 m 步：当前状态为 $x = Z_{m-1}$，提出 proposal $y \sim q(x, \cdot)$，以如下 accept probability 接受 $A(x, y) = \min\left(1, \frac{\tilde{\pi}(y)\, q(y, x)}{\tilde{\pi}(x)\, q(x, y)}\right)$. 若被接受，则令 $Z_m = y$；否则令 $Z_m = x$。

Why MH Works: The MH chain is reversible with respect to π, so π is a stationary distribution. If the chain is also irreducible and aperiodic, then Z_n approaches π in distribution. MH 先保证"目标分布是平稳的"，再加上链本身"能到处走且不周期卡住"，就能保证长期采样结果逼近目标分布。

Random Walk MH: 从当前点 x 加一个随机扰动得到候选点 y。Choose proposal of the form $q(x, y) = q(y - x)$. Common choices: $y - x \sim \mathcal{N}(0, \Sigma)$ or $y - x \sim \mathrm{Unif}[-\delta, \delta]^d$. acceptance rate about 0.25 to 0.5 ***Metropolis Algorithm (Symmetric Proposal)***: 如果提议分布是对称的，即 $q(x, y) = q(y, x)$, then acceptance simplifies to $A(x, y) = \min\left(1, \frac{\tilde{\pi}(y)}{\tilde{\pi}(x)}\right)$. 候选点密度更高就必接收，更低就按比例概率接收

Independence Chain MH: Choose proposal independent of current state: 每一步候选点 y 都直接从同一个分布 q 抽，不管现在在 x 的哪里。$q(x, y) = q(y)$. Works well only if q approximates π well, usually with heavier tails than π. 这样能更稳地覆盖 π 的尾部，避免某些区域几乎提议不到，导致混合变差或接受率很低。acceptance rate close to 1 is preferred.

Proposal Tuning: Random walk MH:



too small variance: slow exploration, high acceptance rate. too large variance:big jumps, low acceptance rate, chain gets stuck.

Gibbs Sampling: 想从多变量分布 $p(z_1, \dots, z_d)$ 中采样. If full conditionals $p(z_i \mid z_{-i})$ are available, iterate:
$z_1^{(k)} \sim p(z_1 \mid z_2^{(k-1)}, \dots, z_d^{(k-1)})$,
$z_2^{(k)} \sim p(z_2 \mid z_1^{(k)}, z_3^{(k-1)}, \dots, z_d^{(k-1)})$,
$z_d^{(k)} \sim p(z_d \mid z_1^{(k)}, \dots, z_{d-1}^{(k)})$.

Why Gibbs Works: Each Gibbs update leaves the target joint distribution invariant. Equivalently, Gibbs is an MH step with acceptance probability 1.

How to Get Full Conditionals: Use $p(z_i \mid z_{-i}) = \frac{p(z_1, \dots, z_d)}{p(z_{-i})}$. In practice: start from the joint density, ignore all terms not depending on z_i, and identify the resulting distribution.

Gibbs Example:
1. For $p(x, y, n) \propto \binom{n}{x} y^{x+a-1} (1-y)^{n-x+b-1} e^{-\lambda} \frac{\lambda^n}{n!}$, the full conditionals are $x \mid y, n \sim \mathrm{Bin}(n, y)$, $y \mid x, n \sim \mathrm{Beta}(x + a, n - x + b)$, $n \mid x, y \sim \mathrm{Pois}((1-y)\lambda) + x$.
2. $p(z_1, z_2, z_3) \propto z_1^{z_2+a-2} (1-z_1)^{b-1} \frac{1}{z_2!} \mathbb{1}_{\{z_3 \in [0, z_1]\}}$.
$p(z_1 \mid z_2, z_3) \propto z_1^{z_2+a-2} (1-z_1)^{b-1} \mathbb{1}_{\{z_1 \geq z_3\}}$, so $z_1 \mid z_2, z_3 \sim \mathrm{Beta}(z_2 + a - 1, b)$ truncated to $[z_3, 1]$.
$p(z_2 \mid z_1, z_3) \propto \frac{z_1^{z_2}}{z_2!}$, so $z_2 \mid z_1, z_3 \sim \mathrm{Pois}(z_1)$.
$p(z_3 \mid z_1, z_2) \propto \mathbb{1}_{\{0 \leq z_3 \leq z_1\}}$, so $z_3 \mid z_1, z_2 \sim \mathrm{Unif}(0, z_1)$.

Initialize $(z_1^{(0)}, z_2^{(0)}, z_3^{(0)})$. For each iteration k, update: $z_1^{(k)}, z_2^{(k)}, z_3^{(k)}$
Poisson distribution is $\mathbb{P}(X = k) = \frac{e^{-\lambda} \lambda^k}{k!}$.

Ways to Obtain Approximately i.i.d. Samples from Gibbs: 因为 Gibbs 连续产生的样本通常有相关性，不是严格独立，所以用两种常见策略减弱相关性：Option 1: run r independent chains and keep the final sample from each. Option 2: run one long chain, discard burn-in, then thin by keeping every dth sample.

Gibbs for Binary Hidden States: Gibbs sampling 用于从 posterior $p(z \mid x)$ 中采样 hidden states。每次固定其他 variables，只更新一个 z_i: $z_i \sim p(z_i \mid z_{-i}, x)$.
Full conditional: $p(z_i \mid z_{-i}, x) \propto$ terms involving z_i.
实际操作：从 joint distribution 中只保留和 z_i 有关的 terms，其他 terms 当作 constants 忽略。
If $z_i \in \{D, U\}$, compute two unnormalized weights: $w_D = P(z_i = D) P(x_i \mid z_i = D) \psi(D, z_{i-1}, z_{i+1})$, $w_U = P(z_i = U) P(x_i \mid z_i = U) \psi(U, z_{i-1}, z_{i+1})$.
Then normalize: $P(z_i = D \mid z_{-i}, x) = \frac{w_D}{w_D + w_U}$, $P(z_i = U \mid z_{-i}, x) = \frac{w_U}{w_D + w_U}$.

Artificial Neuron: Local field / pre-activation: $v = \sum_{j=1}^m w_j x_j + b$. Bias as an additional input: $v = \sum_{j=0}^m w_j x_j$, with $x_0 = 1$ and $w_0 = b$. Neuron output: $y = f(v)$.

Common Activation Functions: Sigmoid: $f(x) = \frac{1}{1+e^{-x}}$. Threshold / step function: $\sigma(v) = 1$ if $v \geq 0$, and 0 otherwise. ReLU: $\mathrm{ReLU}(x) = \max(0, x)$.

Single-Neuron Perceptron: Decision rule: $y = 1$ if $w^\top p + b > 0$, otherwise $y = 0$. Decision boundary: $w^\top p + b = 0$. The weight vector w is orthogonal to the boundary. Positive decision region: $w^\top p + b > 0$.

Decision Boundary Geometry: All points on the boundary satisfy the same inner product: $w^\top p = -b$. Scaling by any nonzero constant does not change the boundary: $w^\top p + b = 0 \iff c\, w^\top p + c\, b = 0$.

OR Perceptron Example: A valid choice is $w = \begin{bmatrix} 1 \\ 1 \end{bmatrix}, b = -0.5$. Then $w^\top p + b = p_1 + p_2 - 0.5$, which implements OR with a threshold neuron.

AND Perceptron Example: A valid choice is $w = \begin{bmatrix} 1 \\ 1 \end{bmatrix}, b = -1.5$. Then $w^\top p + b = p_1 + p_2 - 1.5$, which implements AND with a threshold neuron.

Perceptron Limitation: A single-layer perceptron can represent only a linear decision boundary, so it cannot solve linearly inseparable problems such as XOR.

XOR with a Multi-Layer Network: One construction uses two hidden neurons: $h_1 = \sigma\left(\begin{bmatrix} 1 & 1 \end{bmatrix} p - 0.5\right)$, $h_2 = \sigma\left(\begin{bmatrix} -1 & -1 \end{bmatrix} p + 1.5\right)$. Output neuron: $y = \sigma(h_1 + h_2 - 1.5)$.

Multi-Layer Perceptron (MLP) Forward Pass: For a 3-layer network, $h_1 = \sigma(W_1 p + b_1)$, $h_2 = \sigma(W_2 h_1 + b_2)$, $\hat{y} = \sigma(W_3 h_2 + b_3)$. Dimension reminder: if $h_1 \in \mathbb{R}^4$, $h_2 \in \mathbb{R}^2$, $\hat{y} \in \mathbb{R}$, then $W_1 \in \mathbb{R}^{4 \times d}$, $W_2 \in \mathbb{R}^{2 \times 4}$, $W_3 \in \mathbb{R}^{1 \times 2}$.

Universal Approximation / Decision Regions: A 1-layer network gives a linear decision boundary. A 2-layer network can generate a single convex decision region. A 3-layer MLP can approximate arbitrary decision regions with enough hidden neurons.

Universal Approximation Theorem (informal): A 2-layer MLP with nonconstant, bounded, and continuous hidden neurons, plus a linear output neuron, can approximate any continuous function arbitrarily well if it has enough hidden neurons.

Forward Propagation: For one layer, $a^{(l)} = \sigma\left(W^{(l)} a^{(l-1)} + b^{(l)}\right)$. With input $a^{(0)} = x$, an L-layer MLP computes $y = f(x) = a^{(L)} = \sigma\left(W^{(L)} \sigma(W^{(L-1)} \cdots \sigma(W^{(1)} x + b^{(1)}) \cdots + b^{(L-1)}) + b^{(L)}\right)$.

Network Parameters: $\theta = \{W^{(1)}, b^{(1)}, W^{(2)}, b^{(2)}, \dots, W^{(L)}, b^{(L)}\}$.

Softmax Output Layer: For logits $z = (z_1, \dots, z_K)$, $\hat{y}_k = \frac{e^{z_k}}{\sum_{j=1}^K e^{z_j}}, 0 < \hat{y}_k < 1, \sum_{k=1}^K \hat{y}_k = 1$.

Common Activation Functions: Sigmoid: $\sigma(x) = \frac{1}{1+e^{-x}}$. Tanh: $\tanh(x) = 2\sigma(2x) - 1$. ReLU: $\mathrm{ReLU}(x) = \max(0, x)$. Leaky ReLU: $\mathrm{LReLU}(x) = \begin{cases} \alpha x, & x < 0, \\ x, & x \geq 0. \end{cases}$ Linear / identity: $f(x) = cx$ (often $c = 1$).used in regression output layer. Also known as identity activation function.

Data Preprocessing: Standardization: $x_{\text{std}} = \frac{x - \mu}{\sigma}$. Min-max normalization: $x_{\text{norm}} = \frac{x - x_{\min}}{x_{\max} - x_{\min}}$.

Classification Targets: For K classes, use K output neurons and one-hot targets: $y = (0, \dots, 0, 1, 0, \dots, 0)$. Prediction: $\hat{c} = \arg\max_k \hat{y}_k$.

Training Objective: For dataset $\{(x^{(i)}, y^{(i)})\}_{i=1}^N$, $\mathcal{L}(\theta) = \sum_{i=1}^N \mathcal{L}_i(\theta)$, or often the average loss $\frac{1}{N} \sum_{i=1}^N \mathcal{L}_i(\theta)$.

Cross-Entropy Loss: For multi-class classification, $\mathcal{L}(\theta) = -\frac{1}{N} \sum_{i=1}^N \sum_{k=1}^K y_k^{(i)} \log \hat{y}_k^{(i)}$. 其中 N 是 samples 数量，K 是 classes 数量，$y_k^{(i)}$ 是 true label，$\hat{y}_k^{(i)}$ 是 predicted probability。
Binary form used in the slides: $\mathcal{L}(\theta) = -\frac{1}{N} \sum_{i=1}^N \sum_{k=1}^K \left(y_k^{(i)} \log \hat{y}_k^{(i)} + (1 - y_k^{(i)}) \log(1 - \hat{y}_k^{(i)})\right)$
对于 binary classification，$y \in \{0, 1\}$，$\hat{y} = P(y = 1 \mid x)$。若 $y = 1$，loss 变成 $-\log \hat{y}$；若 $y = 0$，loss 变成 $-\log(1 - \hat{y})$。

Mean Squared Error (MSE): For regression, $\mathcal{L}(\theta) = \frac{1}{N} \sum_{i=1}^N (y^{(i)} - \hat{y}^{(i)})^2$. Mean absolute error (MAE): $\frac{1}{n} \sum_i |y_i - \hat{y}_i|$

Gradient Descent (GD): Gradient: $\nabla \mathcal{L}(\theta) = \left[\frac{\partial \mathcal{L}}{\partial \theta_i}\right]_i$. Update rule: $\theta_{\text{new}} = \theta_{\text{old}} - \alpha \nabla \mathcal{L}(\theta_{\text{old}})$, where α is the learning rate.

Backpropagation: Each training step uses: forward pass → compute loss → backward pass → update parameters. Backprop computes $\nabla \mathcal{L}(\theta)$ using the chain rule.

Mini-Batch Gradient Descent: For a mini-batch $\mathcal{B}$, $\mathcal{L}_{\mathcal{B}}(\theta) = \frac{1}{|\mathcal{B}|} \sum_{i \in \mathcal{B}} \mathcal{L}_i(\theta)$, and update using $\nabla \mathcal{L}_{\mathcal{B}}(\theta)$. Typical batch size: 32 to 256.

Stochastic Gradient Descent (SGD): SGD uses a mini-batch of size 1. In practice, "SGD"in DL libraries usually means mini-batch GD, often with momentum.

Gradient Descent with Momentum: Use running velocity v_t: $v_t = \beta v_{t-1} - \alpha \nabla \mathcal{L}(\theta_{t-1})$, $\theta_t = \theta_{t-1} + v_t$.

Nesterov Momentum: Compute the gradient at a look-ahead point: $v_t = \beta v_{t-1} - \alpha \nabla \mathcal{L}(\theta_{t-1} + \beta v_{t-1})$, $\theta_t = \theta_{t-1} + v_t$.

Adam: Adam uses moving averages of gradients and squared gradients: $m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$, $v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$. Commonly used together with bias correction and adaptive updates.

Learning Rate: α controls the step size of parameter updates. Too small ⇒ very slow convergence. Too large ⇒ unstable training, divergence, saturated activations, or dead ReLU units.

Learning Rate Scheduling: Annealing / decay reduces α over time to make training more stable. Warmup gradually increases α at the beginning, then decays it later. Common schedules: step decay, exponential decay, cosine decay, reduce-on-plateau.

Vanishing / Exploding Gradients: Very small gradients cause weights to update very slowly or stop updating. Very large gradients cause unstable updates or divergence. This is common in very deep networks due to repeated chain-rule multiplication. Common remedies: proper initialization, ReLU-based activations, Batch Normalization, residual connections, and gradient clipping.

Saturated Activations: Sigmoid and tanh can saturate when inputs are very large or very small, causing gradients to become close to zero. This leads to slow learning or weights stopping updates. Common remedies: Xavier initialization, Batch Normalization, input normalization, and using ReLU-based activations.

Dead ReLU: A ReLU unit is dead if its input is always negative, so its output and gradient are always zero. The weights connected to this unit may stop updating. Common remedies: smaller learning rate, He initialization, Leaky ReLU, ELU, or Batch Normalization.

Generalization: Underfitting: high training error and high validation error. Overfitting: low training error but high validation error.

Model Capacity: More neurons / more layers ⇒ higher capacity. Too little capacity ⇒ underfitting. Too much capacity ⇒ higher overfitting risk and harder optimization.

ℓ_2 Regularization: $\mathcal{L}_{\text{reg}}(\theta) = \mathcal{L}(\theta) + \lambda \sum_k \theta_k^2$.

ℓ_1 Regularization: $\mathcal{L}_{\text{reg}}(\theta) = \mathcal{L}(\theta) + \lambda \sum_k |\theta_k|$.

Elastic Net: $\mathcal{L}_{\text{reg}}(\theta) = \mathcal{L}(\theta) + \lambda_1 \sum_k |\theta_k| + \lambda_2 \sum_k \theta_k^2$.

Dropout: Randomly drop units during training to reduce overfitting. Let p be the probability that a unit is retained: $\tilde{h}_i = m_i h_i$, $m_i \sim \mathrm{Bernoulli}(p)$. Disable dropout during evaluation and use all units, scaling by p: $\tilde{h}_i = p h_i$. Equivalently, with inverted dropout, use $\tilde{h}_i = \frac{m_i h_i}{p}$ during training and $\tilde{h}_i = h_i$ during evaluation.

Early Stopping: Use a validation set and stop training when validation loss / accuracy does not improve for n epochs. The hyper-parameter n is called *patience*.

Batch Normalization: For a batch input x with mean μ and standard deviation σ, $\hat{x} = \frac{x - \mu}{\sigma}$ (标准化) $x_{\text{BN}} = \gamma \hat{x} + \beta$.(把标准化后的值进行缩放和平移) Here, γ and β are learnable scale and shift parameters. In CNNs, BatchNorm usually has $2C$ learnable parameters for C channels. It does not change the activation shape.
Benefits: stabilizes training, improves gradient flow, allows larger learning rates, and reduces sensitivity to initialization.

Hyper-Parameter Tuning: • layer size+num neur, LR(schedule), optimizer type, regularization(L2), batch size, activation/loss function.
• Methods: grid search(check all with step), random search(prefer), Bayesian optimization. Use k-fold cross-validation if data is limited.

k-Fold Cross-Validation: Split training data into k folds. Train on $k - 1$ folds, validate on the remaining fold, repeat for all folds, then average validation results. Used manly when training data is limited.

Deep vs. Shallow Networks: Deeper networks often perform better than shallow ones, but only up to a limit; performance may plateau after too many layers.

Residual Networks (ResNets)
• Use identity=skip connections(out=in +sigma) • prevent vanishing gradients. • very deep models(18, 50, 152 layers).

From MLP to CNN: MLPs are not ideal for images because they flatten the image into a vector, which ignores the original spatial structure and makes the model sensitive to object position. The same pattern shifted to a different location may produce a very different output. CNNs are more suitable for image tasks because they use local connectivity to extract local features such as edges, corners, and strokes, and use weight sharing to apply the same filters across different positions. This preserves spatial structure, reduces the number of parameters, and makes CNNs more efficient and less prone to overfitting.

Convolution:
1D Continuous form: $(x * w)(t) = \int_{-\infty}^{\infty} x(\tau)\, w(t - \tau)\, d\tau$
Discrete form: $(x * w)[n] = \sum_{k=-\infty}^{\infty} x[k]\, w[n - k]$
egample: $x = [1, 2, 3, 4]$, $w = [5, 6, 7]$，卷积后得到 $z = [5, 16, 34, 52, 45, 28]$
2D discrete form: $(x * w)[m, n] = \sum_{i=-\infty}^{\infty} \sum_{j=-\infty}^{\infty} x[i, j]\, w[m - i,\ n - j]$

Local receptive field:In CNNs, neurons in a layer are only connected to a small spatial region of the previous layer.

Pooling:
Max pooling and average pooling.Pooling layers reduce the spatial size of the feature maps. Reduce the number of parameters, prevent overfitting.

CNNs learn hierarchical features: early layers detect edges and textures, middle layers detect parts and patterns, and deeper layers capture semantic features for final classification.

Convolution Output Size and Zero Padding:
$N_{\text{out}} = \frac{N - F}{S} + 1$, $N_{\text{out,pad}} = \frac{N + 2P - F}{S} + 1$
For odd filter size F and stride $S = 1$, choose $P = \frac{F-1}{2}$ so that $N_{\text{out,pad}} = N$. Without padding, repeated convolutions shrink spatial size (e.g. 32 → 28 → 24)Shrinking too fast is not good.

Example:
Input: $32 \times 32 \times 3$, $K = 10$, $F = 5$, $S = 1$, $P = 2$
$N_{\text{out}} = \frac{N + 2P - F}{S} + 1 = \frac{32 + 2 \cdot 2 - 5}{1} + 1 = 32$
Output volume: $32 \times 32 \times 10$
Params/filter: $F \cdot F \cdot C_{\text{in}} + 1 = 5 \cdot 5 \cdot 3 + 1 = 76$
Total params: $K \cdot 76 = 10 \cdot 76 = 760$

Code Example:
```
conv1 = torch.nn.Conv2d(in_channels=3,
out_channels=2,
kernel_size=3, stride=1, padding=0)
print(conv1(x.unsqueeze(0)).shape) # add
batch dimension
```

Pooling layer:
1. Makes the representations smaller and more manageable.
2. Operates over each activation map independently.
Assume input is W1 x H1 x C, Conv layer needs 2 hyperparameters:
1. The spatial extent (filter size) F, pooling window 的大小
2. The stride S
Output W2 x H2 x C:
W2 = (W1 - F)/S + 1, H2 = (H1 - F)/S + 1
Number of parameters: 0
```
pool1 = torch.nn.MaxPool2d(kernel_size=2,
stride=2, padding=0)
# Commonly, stride == kernel_size
```

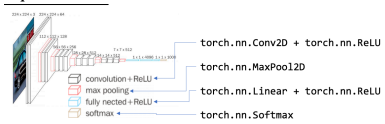
Flatten: Flatten only changes shape, not values. It converts spatial feature maps into vectors for FC/Linear layers.

Layers with no parameters: Input, activation layers such as ReLU, Leaky ReLU, sigmoid and tanh, pooling layers, dropout, flatten, and softmax layers have no learnable parameters. They may have hyperparameters, such as pooling size, stride, or dropout probability, but these are not learned during training.

Flatten example:
Input volume: $8 \times 8 \times 16$ Flatten output size: $8 \cdot 8 \cdot 16 = 1024$
Output: 1024 Total params: 0

Fully-connected layer example:
Input size: 1024, number of output neurons: 10
Weights: $1024 \cdot 10$ Biases: 10
Total params: $1024 \cdot 10 + 10 = 10250$ Output: 10

Implementation:



Architecture pattern:
[(CONV-RELU)* N - POOL?]* M - (FC-RELU) * K - SOFTMAX
```
model = nn.Sequential(
nn.Conv2d(in_channels=3, out_channels=32,
kernel_size=3, stride=1, padding=1),
nn.ReLU(),
nn.MaxPool2d(kernel_size=2, stride=2),
nn.Flatten(),
nn.Linear(in_features=..., out_features=64),
nn.ReLU(),
nn.Linear(in_features=64,
out_features=num_classes),
nn.Softmax(dim=1))
```

Gradient Descent (GD)
1. **Initialization:** Randomly initialize the model parameters θ^0. Set $\theta^{\text{old}} = \theta^0$.
2. Compute the gradient of the loss function at θ^{old}: $\nabla \mathcal{L}(\theta^{\text{old}})$.
3. Update the parameters: $\theta^{\text{new}} = \theta^{\text{old}} - \alpha \nabla \mathcal{L}(\theta^{\text{old}})$ where α is the learning rate.
4. repeat until terminating.
GD does not guarantee reaching a global minimum.
• Backpropagation: Combines forward pass + backward pass using chain rule to compute gradients of loss w.r.t. weights.
• Batch GD: compute gradients over full dataset.
• Mini-batch GD: over subset (32 to 256 samples). Mini-batch gradient approximates the full training set gradient well.
• SGD: mini-batch size = 1, fast but noisy. Less used.

GD with Momentum:
• Momentum: $v_t = \beta v_{t-1} + (1 - \beta) \nabla \mathcal{L}$, improves convergence.
• Nesterov Momentum: lookahead at next step before computing gradient. (Skip)
• Adam: computes a weighted average of past gradients(1st moment of G) and weighted average of past squared gradients(2rd). • Other: RMSprop, Adagrad, Adadelta, Nadam, etc. • Most used: Adam, SGD with momentum.